



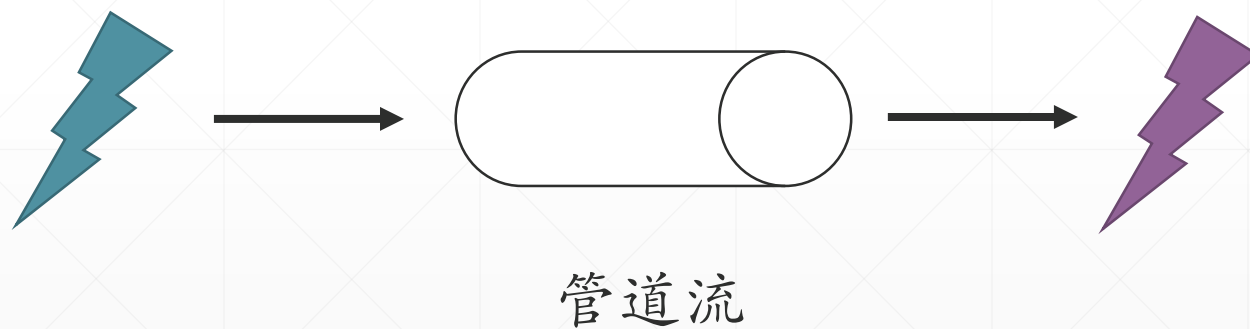
管道流

北京理工大学计算机学院
金旭亮

概述



管道流可用于线程之间的通讯：



管道流使用示例

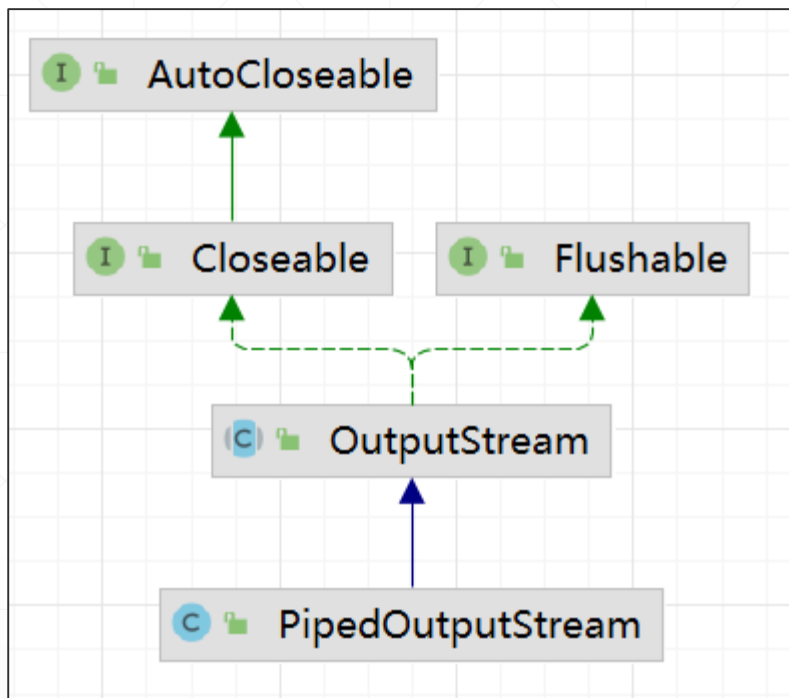
```
public class PipedDemo {  
    public static void main(String[] args) throws IOException {  
        var pis = new PipedInputStream();  
        var pos = new PipedOutputStream();  
        //创建管道  
        pos.connect(pis);  
        //启动两个线程, 进行通讯  
        Producer p = new Producer(pos);  
        Consumer c = new Consumer(pis);  
        p.start();  
        c.start();  
    }  
}
```

Run: PipedDemo x

Consumer 97
Consumer 98
Consumer 99
Consumer 100
收到的数累加和: 5050

Process finished with exit code 0

数据生产者



数据生产者封装了一个从外部注入的 PipedOutputStream 对象，在管道建立之后，调用它的 write 方法发送数据.....

```
class Producer extends Thread {
    OutputStream os;
    public Producer(OutputStream o) {
        os = o;
    }
    public void run() {
        int count = 1;
        while (count <= 100) {
            try {
                os.write(count);
                os.flush();
                System.out.println("Producer " + count);
                System.out.flush();
                Thread.sleep(10);
                count++;
            } catch (Exception e) {
            }
        }
        try {
            os.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

注意这个数值

管道输入流组件所提供的方法

InputStream		
m	skipNBytes(long)	void
m	readAllBytes()	byte[]
m	available()	int
m	transferTo(OutputStream)	long
m	skip(long)	long
m	reset()	void
m	markSupported()	boolean
m	read(byte[], int, int)	int
m	nullInputStream()	InputStream
m	read(byte[])	int
m	readNBytes(int)	byte[]
m	close()	void
m	mark(int)	void
m	read()	int
m	readNBytes(byte[], int, int)	int

PipedInputStream		
m	connect(PipedOutputStream)	void
m	initPipe(int)	void
m	close()	void
m	receivedLast()	void
m	checkStateForReceive()	void
m	available()	int
m	read()	int
m	awaitSpace()	void
m	receive(int)	void
m	receive(byte[], int, int)	void
m	read(byte[], int, int)	int



数据消费者

```
class Consumer extends Thread {  
    InputStream is;  
    public Consumer(InputStream s) {  
        is = s;  
    }  
}
```

消费者封装了PipedInputStream。
注意，尽管数据的发送与消费的
速率不匹配，但由于管道“一次
只能发送一个”的特性，所以，
示例最后结果还是正确的。

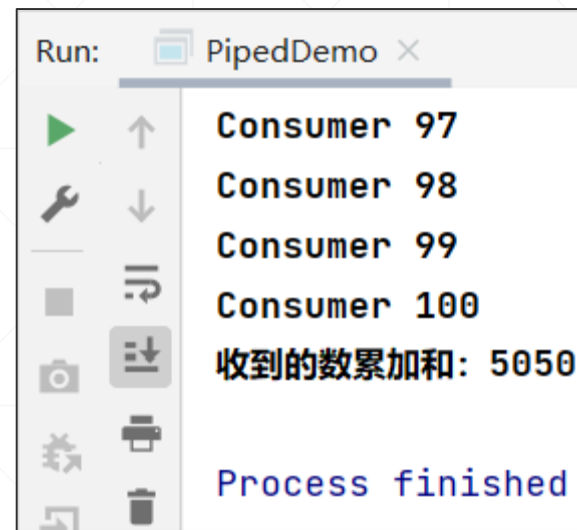
```
public void run() {  
    int x = 0, sum = 0;  
    do {  
        try {  
            x = is.read();  
            if (x != -1) {  
                System.out.println("Consumer " + x);  
                sum += x;  
            }  
            System.out.flush();  
            //处理时间与发送时间不一致，会导致数据积压或等待  
            Thread.sleep(millis: 50);  
        } catch (IOException | InterruptedException e) {  
            e.printStackTrace();  
        }  
    } while (x != -1);  
    try {  
        System.out.println("收到的数累加和: " + sum);  
        is.close();  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

运行结果测试分析

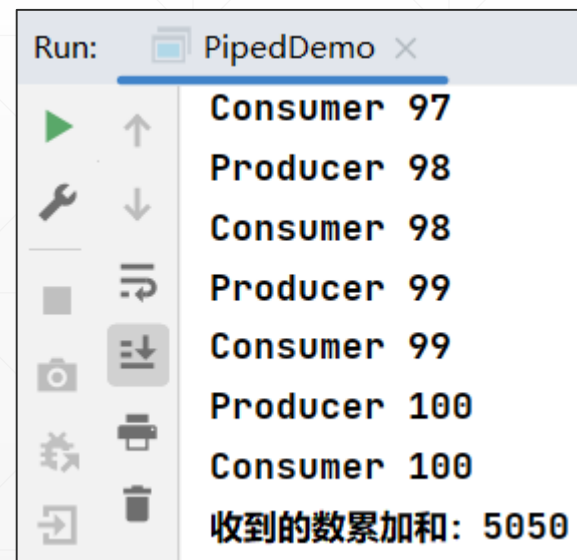
```
pos.connect(pis);
```

当“生产者”速率（10毫秒间隔）快于“消费者”速率（50毫秒间隔）时，会导致“数据”在发送端积压，因此示例中会在后面连续输出多个“Consumer ...”。

当“生产者”速率（100毫秒间隔）慢于“消费者”速率（50毫秒间隔）时，会导致“数据”在接收端等待，因此示例中“Consumer ...”与“Producer ...”是交替出现的。



```
Run: PipedDemo x
Consumer 97
Consumer 98
Consumer 99
Consumer 100
收到的数累加和: 5050
Process finished
```



```
Run: PipedDemo x
Consumer 97
Producer 98
Consumer 98
Producer 99
Consumer 99
Producer 100
Consumer 100
收到的数累加和: 5050
```

提示



管道适合于两个线程之间快速地传送大量的数据，不过，要求发送方与接收方的数据处理速度相差不要太大，否则，有可能会因为发送与接收的速度不匹配数据在一端“积压”或“阻塞等待”，带来问题。



RxJava对这个问题有比较成熟的解决方案，提供了一种称为“back pressure（背压）”的特性实现流量控制，感兴趣的同學可以学习金老师教学网站上的“Java异步与反应式编程入门”这门在线课程。